

IA PARA DESENVOLVEDORES [T1]

Situação de Aprendizagem (Projeto Avaliativo) - Módulo 2 -
Semana 12

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	3
4. REQUISITOS DA APLICAÇÃO	4
4.1. Domínio, escopo e cenários	4
4.2. Arquitetura agêntica e LangGraph	4
4.3. Tools, MCP e integrações	4
4.4. Memória, contexto e RAG	4
4.5. Segurança, governança e limites de autonomia	5
4.6. Observabilidade e resiliência	5
4.7. IA para QA e testes inteligentes	5
4.8. DevOps inteligente e detecção de falhas	5
4.9. Low-Code para QA, SRE e agentes	6
4.10. Prompts, modelos e refinamento	6
5. ROTEIRO DA APLICAÇÃO	6
5.1. FORMATO DO SISTEMA	7
5.2. DOCUMENTAÇÃO NO README.MD	7
5.3. USO DO QUADRO DO GITHUB	8
5.4. USO DO REPOSITÓRIO NO GITHUB	9
5.5. GRAVAÇÃO DE VÍDEO	9
6. CRITÉRIOS DE AVALIAÇÃO	10
7. CHECKLIST FINAL DE ENTREGA	14

1. CONTEXTUALIZAÇÃO

Sistemas de inteligência artificial deixaram de atuar apenas como assistentes conversacionais e passaram a executar tarefas, consultar dados, utilizar ferramentas, manter memória e coordenar fluxos de múltiplas etapas. Essa evolução amplia o valor da IA no desenvolvimento de software, mas também introduz novos riscos: uma ferramenta pode receber parâmetros inválidos, um loop pode não terminar, uma memória pode recuperar contexto inadequado, uma ação pode ultrapassar o

nível de autonomia permitido e uma execução aparentemente correta pode esconder falhas, tentativas repetidas ou decisões sem rastreabilidade.

Ao longo do Módulo 2, foram estudados os fundamentos de agentes, a diferença entre assistentes, agentes e workflows, o uso de tools e MCP, a construção de fluxos com LangGraph, memória curta e longa, RAG, integrações com APIs, segurança, governança, automação de CI/CD e ChatOps, arquitetura escalável, observabilidade, revisão de código, testes inteligentes, detecção de falhas e automações low-code.

Neste projeto avaliativo, você deverá desenvolver uma aplicação funcional em qualquer domínio de negócio, desde que o uso de IA esteja presente na solução e seja demonstrado nas atividades previstas neste projeto. Você poderá dar continuidade à aplicação desenvolvida no mini-projeto do módulo, evoluindo-a de forma coerente e sem a necessidade de criar um sistema excessivamente grande.

O objetivo não é acumular funcionalidades, mas construir uma solução funcional, demonstrável e tecnicamente explicável. O projeto deverá evidenciar como a entrada percorre o fluxo, como o agente toma decisões, quais tools são utilizadas, como o contexto é mantido ou recuperado, quais controles impedem comportamentos inseguros, como a qualidade é verificada e quais sinais permitem investigar o funcionamento da solução.

2. DESAFIO

Você deverá desenvolver ou evoluir uma aplicação funcional com agentes de IA, em qualquer domínio de negócio, integrando os conteúdos essenciais do Módulo 2. A solução deverá receber uma solicitação, evento ou conjunto de dados, executar um fluxo com múltiplas etapas, utilizar ferramentas ou serviços, produzir uma saída estruturada e registrar evidências suficientes para que outra pessoa consiga compreender, testar e reproduzir seu comportamento.

A solução poderá ser derivada do mini-projeto já desenvolvido. Nesse caso, você deverá demonstrar claramente quais capacidades foram mantidas, quais foram refatoradas e quais evoluções foram adicionadas para atender aos requisitos deste projeto final.

Diante disso, você deverá:

- Definir um problema real ou plausível, seus usuários, entradas, saídas, riscos e critérios de sucesso;

- Explicar se a solução é um agente, um workflow determinístico ou um sistema híbrido, justificando essa classificação;
- Modelar o fluxo principal com LangGraph, utilizando estado compartilhado, nodes, edges e controle explícito de execução;
- Implementar pelo menos uma tool funcional integrada por MCP, API, serviço, backend ou webhook, com validação de entradas e tratamento de erros;
- Implementar estratégia de memória e recuperação contextual adequada ao domínio;
- Aplicar segurança, governança, limites de autonomia, controle de looping e aprovação humana quando necessária;
- Produzir e correlacionar pelo menos dois sinais de observabilidade que permitam investigar e reconstruir uma execução;
- Aplicar IA em revisão de código, geração ou refinamento de testes e priorização orientada a risco;
- Integrar práticas de DevOps inteligente, incluindo explicação de logs, detecção de anomalias e estimativa de tendência ou risco de falha;
- Adicionar ao menos uma integração low-code ou no-code para QA, SRE, automação ou construção visual de agentes;
- Documentar prompts, decisões, limitações, testes e evidências de execução.

3. RESULTADOS ESPERADOS (ENTREGA)

A atividade será realizada individualmente. Cada estudante será responsável pelo planejamento, desenvolvimento, documentação, testes, organização do GitHub e demonstração da própria solução, mantendo evidências reais da evolução do trabalho por meio de cards, commits, branches, pull requests, testes, prompts e demais artefatos produzidos.

O código deverá ser inserido e versionado em um repositório no GitHub, que poderá ser criado na conta pessoal do estudante, e o planejamento deverá ser realizado em um GitHub Project no formato Kanban. O professor deverá receber acesso aos artefatos necessários para a avaliação. Os links do repositório, do quadro e do vídeo deverão ser submetidos na atividade correspondente do AVA.

A entrega será composta pelos seguintes artefatos principais:

- Repositório no GitHub com código, testes, workflows, documentação e histórico de desenvolvimento;
- Quadro Kanban do GitHub atualizado durante o projeto;

- README.md completo, com instruções de instalação, configuração, execução e descrição da solução;
- Documentação e evidências técnicas necessárias para demonstrar as principais decisões, refinamentos, testes, observabilidade, análise de falhas e integração low-code/no-code;
- Vídeo de demonstração publicado no YouTube como não listado.

Peso deste projeto: Avaliação M2.2 – 60% da nota do módulo.

Data de liberação: 21/08/26 às 22h

Data de entrega: 31/08/26 até às 15h

Submissão no AVA: Projeto Avaliativo – M2.2

4. REQUISITOS DA APLICAÇÃO

O desenvolvimento da aplicação funcional pode ser feito em qualquer domínio de negócio, desde que o uso de IA esteja presente na solução e seja demonstrado nas atividades previstas neste projeto. As tecnologias poderão ser definidas de acordo com o domínio escolhido. A avaliação considerará a coerência entre o problema, a arquitetura, o nível de autonomia, os mecanismos de qualidade e as evidências apresentadas.

4.1. Domínio, escopo e cenários

- O problema, o público, as entradas, as saídas e os limites da solução deverão estar descritos no README.md.
- A aplicação deverá possuir lógica funcional compatível com o problema escolhido, sem depender exclusivamente de respostas fixas no código.
- Deverão ser demonstrados pelo menos dois cenários de uso, sendo um fluxo principal e um cenário de risco, falha, exceção ou comportamento anômalo.
- A saída principal deverá ser estruturada e adequada ao domínio, podendo utilizar JSON, modelo Pydantic, tabela, relatório, contrato de API ou formato equivalente.

4.2. Arquitetura agêntica e LangGraph

- Implementar o fluxo principal com LangGraph, utilizando estado compartilhado tipado, nodes com responsabilidades claras e edges explícitas.
- O fluxo deverá contemplar execução sequencial, ramificação condicional e ao menos uma paralelização simples.

- Definir condições de continuidade e parada, evitando loops indefinidos e execuções desnecessárias.
- Manter clara a separação entre decisões realizadas pelo modelo e regras determinísticas da aplicação.

4.3. Tools, MCP e integrações

- Implementar pelo menos uma tool funcional, com entradas e saídas bem definidas, integrada por MCP, API, serviço, backend ou webhook, incluindo validação de payloads, parâmetros e schemas e tratamento de falhas;
- Ações destrutivas ou irreversíveis deverão ser simuladas, bloqueadas ou condicionadas à aprovação humana, quando aplicável ao domínio.

4.4. Memória, contexto e RAG

- Implementar uma estratégia de memória ou recuperação contextual adequada ao domínio da aplicação, utilizando recursos como state, checkpoint, armazenamento persistente ou RAG.
- A estratégia adotada deverá permitir que a solução utilize informações relevantes da própria execução, de interações anteriores ou de uma fonte externa, conforme a necessidade do domínio.
- Quando utilizar RAG, documentar resumidamente a base, o chunking, a indexação, a recuperação e as fontes. Para outras fontes externas, indicar a origem das informações e como são recuperadas.

4.5. Segurança, governança e limites de autonomia

- Proteger credenciais e informações sensíveis, mantendo segredos fora do repositório, e validar permissões antes da execução de tools ou ações externas.
- Definir limites de autonomia coerentes com o domínio, determinando quando uma ação poderá ser executada, bloqueada ou depender de aprovação humana.
- Implementar e demonstrar pelo menos um cenário adversarial envolvendo prompt injection ou entrada não confiável, comprovando que conteúdos externos não substituem as regras da aplicação, ações não autorizadas não são executadas e informações sensíveis não são reveladas.

4.6. Observabilidade e resiliência

- Produzir e correlacionar pelo menos dois sinais de observabilidade, sendo um deles logs estruturados e o outro podendo ser trace, métrica ou registro de auditoria;
- Utilizar esses sinais para investigar pelo menos uma execução da aplicação, permitindo identificar seu fluxo, decisões relevantes, erros e latência, quando disponível.
- Aplicar tratamento básico de falhas nas integrações externas, como timeout, retry limitado ou fallback, quando aplicável ao domínio.

4.7. IA para QA e testes inteligentes

- Utilizar IA para analisar pelo menos uma alteração real do projeto, como um diff, trecho de código ou Pull Request real, identificando possíveis problemas ou oportunidades de melhoria;
- Gerar ou refinar testes automatizados com apoio de IA, cobrindo cenários relevantes da aplicação e incluindo pelo menos um dos seguintes tipos de teste: integração, aceitação ou E2E (end-to-end).
- Selecionar e justificar pelo menos um teste ou cenário considerado prioritário com base em risco, impacto ou criticidade.

4.8. DevOps inteligente e detecção de falhas

- Configurar um pipeline que execute lint, testes e build ou validação equivalente. O deploy da aplicação não será obrigatório;
- Utilizar IA para analisar e explicar logs de pelo menos duas etapas entre CI, Dockerfile, lint, testes, build e, quando houver, CD ou deploy;
- Detectar e explicar pelo menos uma anomalia, como erro recorrente, latência alta, falha de tool ou aumento da taxa de erro;
- Produzir uma estimativa simples de tendência, risco ou probabilidade de falha, utilizando dados reais ou simulados e documentados;
- Apresentar as evidências utilizadas e justificar a conclusão obtida na análise da falha ou do risco identificado.

4.9. Low-Code para QA, SRE e agentes

Você deverá implementar uma automação low-code ou no-code integrada à solução principal. Não será necessário reconstruir a aplicação na ferramenta visual.

- O fluxo deverá possuir ao menos um gatilho, integrar-se à aplicação ou a um de seus serviços e produzir uma saída observável, como alerta, relatório, registro, comentário ou resposta;
- A lógica principal deverá permanecer na aplicação, enquanto a ferramenta visual deverá atuar como apoio à orquestração ou integração;
- O fluxo deverá possuir instruções resumidas de reprodução no README.md.

Como extensão opcional, o aluno poderá utilizar ChatOps ou outro mecanismo de notificação para comunicar resultados, alertas ou diagnósticos produzidos pela aplicação, utilizando, por exemplo, Discord, Slack, Microsoft Teams, e-mail, GitHub Issue ou webhook.

4.10. Prompts, modelos e refinamento

- Manter documentadas no projeto as principais instruções de sistema utilizadas pelo agente, incluindo regras de comportamento, objetivos da tarefa, restrições importantes e padrões de resposta esperados, além dos prompts relevantes que orientam o funcionamento da solução;
- Configurar o modelo utilizado por meio de variável de ambiente, evitando credenciais ou informações sensíveis no código;
- Documentar pelo menos um ciclo de refinamento de prompt ou comportamento do agente, apresentando o problema observado, a alteração realizada e o resultado obtido.

5. ROTEIRO DA APLICAÇÃO

A seguir estão os requisitos de organização e apresentação da entrega. A estrutura poderá ser adaptada ao domínio, mas deverá permitir que o avaliador localize rapidamente o código do agente, as integrações, os testes, as políticas, os workflows e as evidências.

5.1. FORMATO DO SISTEMA

Você poderá escolher o formato da aplicação, desde que a solução seja executável, demonstrável e documentada. São formatos aceitáveis:

- Aplicação de linha de comando;
- API local com FastAPI, Flask ou tecnologia equivalente;
- Interface simples com Gradio, Streamlit ou aplicação web;
- Aplicação integrada a serviços externos ou automações.

Outros formatos poderão ser utilizados desde que sejam adequados ao domínio e permitam demonstrar os requisitos do projeto.

Notebooks poderão ser utilizados apenas como apoio à experimentação ou como demonstração auxiliar, não como formato principal da aplicação entregue.

A solução deverá possuir dados de exemplo e instruções suficientes para reproduzir os cenários demonstrados. Credenciais, chaves de API, tokens e outras informações sensíveis não deverão ser incluídas no repositório, no AVA ou nos demais artefatos entregues. Quando necessário, deverá ser disponibilizado um **.env.example** sem valores reais.

5.2. DOCUMENTAÇÃO NO README.MD

Crie um arquivo README.md no repositório do projeto no GitHub para documentar a solução, apresentar as principais decisões técnicas e permitir que outra pessoa compreenda, configure, execute e avalie a aplicação.

O README.md deverá funcionar como guia do projeto e conter obrigatoriamente:

- **Descrição da solução:** nome do projeto, problema resolvido, público, objetivo e valor entregue. Quando houver continuidade do mini-projeto, indicar brevemente quais capacidades foram mantidas ou evoluídas;
- **Classificação e arquitetura:** classificar a solução como agente, workflow determinístico ou sistema híbrido e apresentar um diagrama da arquitetura, destacando o fluxo LangGraph, seus principais nodes, rotas, paralelização e componentes envolvidos;
- **Tool e integração:** descrever a tool implementada e sua integração, como MCP, API, serviço, backend ou webhook, indicando resumidamente sua finalidade no fluxo;
- **Contexto e memória:** explicar a estratégia de memória ou recuperação contextual adotada, como state, checkpointing, armazenamento persistente ou RAG, e como essas informações são utilizadas pela aplicação;
- **Segurança e autonomia:** apresentar os principais controles de segurança, proteção de credenciais, validações, limites de autonomia, bloqueios e aprovação humana quando aplicável, incluindo o comportamento esperado diante de uma entrada adversarial ou prompt injection;
- **Instalação e execução:** fornecer instruções de configuração, instalação, execução e testes, incluindo as variáveis de ambiente necessárias por meio de **.env.example**, sem expor credenciais, tokens ou informações sensíveis;

- **QA, observabilidade e DevOps:** apresentar as principais evidências de qualidade e operação da solução, incluindo testes realizados, análise de código com IA, os sinais de observabilidade utilizados, pipeline, análise de logs, anomalia identificada e estimativa de tendência ou risco de falha;
- **Automação low-code/no-code:** descrever o fluxo integrado à aplicação, indicando seu gatilho, sua relação com a solução principal e a saída produzida. Quando utilizado, também poderá ser apresentado ChatOps ou outro mecanismo de alerta ou notificação;
- **Cenários de uso:** documentar dois cenários, sendo um fluxo principal e um cenário de risco, falha ou exceção, apresentando exemplos de entrada, comportamento esperado e resultado produzido;
- **Análise crítica e limitações:** apresentar pelo menos um refinamento relevante realizado durante o desenvolvimento, indicando o problema observado, a alteração aplicada e o resultado obtido, além das principais limitações, possibilidades de evolução e link do vídeo de demonstração.

5.3. USO DO QUADRO DO GITHUB

Crie um GitHub Project para organizar o desenvolvimento utilizando as colunas Backlog, A Fazer, Em Andamento, Bloqueado, Em Revisão e Concluído. Os cards deverão refletir o processo real, e não ser criados apenas ao final.

Cada card deverá representar uma atividade real do projeto e conter uma descrição clara da tarefa, seu objetivo e o resultado esperado. Quando aplicável, o card poderá ser relacionado a branches, pull requests, testes ou outras evidências produzidas durante o desenvolvimento.

Para apoiar a organização do trabalho, os temas abaixo podem ser utilizados como referência para a criação dos cards:

- definição do problema, escopo e arquitetura da solução;
- implementação do fluxo com LangGraph;
- desenvolvimento da tool e integração;
- implementação de memória, contexto ou RAG;
- segurança, governança e tratamento de entradas adversariais;
- implementação de logs e demais sinais de observabilidade;
- análise de código e criação ou refinamento de testes com IA;
- configuração do pipeline e análise de logs;
- detecção de anomalias e análise de tendência ou risco de falha;
- integração da automação low-code/no-code;

- documentação, README.md, vídeo e preparação da entrega.

O quadro deve refletir de forma consistente o fluxo de desenvolvimento, garantindo coerência entre tarefas, commits, branches e artefatos gerados ao longo do ciclo de implementação.

5.4. USO DO REPOSITÓRIO NO GITHUB

- Utilize um repositório no GitHub para controle de versionamento, inclusive em sua conta pessoal. O histórico deverá permitir identificar claramente a evolução do projeto.
- Adicionar o professor como colaborador, conforme as orientações fornecidas para a avaliação.
- Utilizar as branches main e develop e criar feature branches a partir da develop.
- Relacionar branches, cards e pull requests sempre que possível.
- Criar commits naturalmente, de acordo com a evolução do projeto, utilizando mensagens semânticas, claras, objetivas e coerentes com cada avanço realizado.
- Manter o código final integrado na main e não alterar o repositório após o prazo.
- Não versionar chaves, tokens, senhas, arquivos .env ou dados sensíveis.
- Incluir .env.example, dependências, comandos de execução, testes e workflows.
- Organizar toda a documentação e as evidências do projeto no diretório **/docs**, utilizando subpastas quando necessário, como **/docs/prompts**, **/docs/qa** e **/docs/evidencias**.

Sugestão de branches — feature/langgraph-agente, feature/tool-integracao, feature/memoria-rag, feature/governanca, feature/observabilidade, feature/qa-inteligente, feature/devops-anomalias, feature/low-code e docs/readme-video.

5.5. GRAVAÇÃO DE VÍDEO

O estudante deverá gravar um vídeo com duração recomendada de até 10 minutos, admitindo-se o limite máximo de 12 minutos, publicá-lo no YouTube como não listado, inserir o link no README.md do repositório e submetê-lo no AVA junto com os demais links do projeto. O vídeo deverá priorizar os seguintes pontos:

- Problema, objetivo e classificação da solução;
- Visão resumida da arquitetura e das integrações;
- Dois cenários de uso, sendo um fluxo principal e um cenário de risco, falha, exceção ou comportamento anômalo;
- Evidência de segurança, bloqueio ou aprovação humana, quando aplicável;
- Uma evidência de QA;
- Pipeline, análise de logs, detecção de anomalias e estimativa de tendência ou risco de falha;
- Demonstração resumida da automação low-code/no-code;
- Principais limitações e melhorias futuras;

Sugestão de roteiro:

- 0:00 a 1:00 — problema, objetivo e classificação da solução;
- 1:00 a 2:00 — visão resumida da arquitetura e das integrações;
- 2:00 a 4:00 — dois cenários de uso, sendo um fluxo principal e um cenário de risco, falha, exceção ou comportamento anômalo;
- 4:00 a 5:00 — evidência de segurança, bloqueio ou aprovação humana, quando aplicável;
- 5:00 a 6:00 — uma evidência de QA;
- 6:00 a 8:00 — pipeline, análise de logs, detecção de anomalias e estimativa de tendência ou risco de falha;
- 8:00 a 9:00 — demonstração resumida da automação low-code/no-code;
- 9:00 a 10:00 — principais limitações e melhorias futuras.

6. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. A nota possui variação de 0 (zero) a 10 (dez). As faixas de pontuação foram

padronizadas em 0 e 1,00; 0 / 0,25 / 0,50; ou 0 / 0,25 / 0,75, conforme o peso de cada bloco.

Serão desconsiderados e atribuídos nota 0 (zero) a projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Também poderão receber nota zero os projetos com credenciais expostas, artefatos inacessíveis ou código que o estudante não consiga explicar durante a demonstração.

Apresentação do Projeto			
Nº	Critério de Avaliação	0	1,00
1	O estudante entregou o vídeo de demonstração no YouTube como não listado, com duração recomendada de até 10 minutos e limite máximo de 12 minutos, cobrindo os pontos descritos no item 5.5?	O vídeo não foi entregue, está inacessível, ultrapassa 12 minutos ou não demonstra o funcionamento e as evidências técnicas solicitadas.	O vídeo está acessível, respeita o limite máximo de 12 minutos e cobre de forma clara os pontos previstos no item 5.5.

Uso adequado do quadro do GitHub				
Nº	Critério de Avaliação	0	0,25	0,50
2	O estudante organizou o escopo do projeto em cards no quadro do GitHub?	O quadro não existe, está inacessível ou não possui cards relacionados ao projeto.	Os cards existem, mas representam apenas parte do trabalho ou possuem descrições pouco claras sobre a atividade e o resultado esperado.	Os cards representam de forma coerente as principais atividades do projeto e possuem descrições suficientes para compreender o que foi planejado e realizado.
3	O estudante manteve o quadro atualizado durante o desenvolvimento?	Os cards não foram movimentados ou foram organizados apenas ao final do projeto.	A movimentação ocorreu de forma parcial ou irregular, sem representar claramente a evolução do trabalho.	O andamento dos cards foi atualizado de forma coerente, permitindo acompanhar a evolução real do projeto durante o desenvolvimento.

Uso adequado do GitHub e README.md				
Nº	Critério de Avaliação	0	0,25	0,75
4	O estudante utilizou adequadamente branches, commits e o fluxo de versionamento do projeto?	O repositório não existe, está inacessível ou concentra o desenvolvimento diretamente na main , sem histórico	O repositório utiliza branches, mas apresenta fluxo inconsistente, histórico concentrado em poucas alterações ou commits com	O desenvolvimento utiliza de forma coerente develop , feature branches e main , com commits incrementais e mensagens semânticas,

		que permita compreender a evolução do projeto.	mensagens genéricas e pouco representativas.	claras e relacionadas à evolução real do projeto.
5	O README.md e a documentação permitem compreender, executar e avaliar a solução?	O README.md está ausente ou não fornece informações suficientes para compreender e executar o projeto.	O README.md existe, mas possui instruções incompletas, informações importantes ausentes ou documentação insuficiente para compreender plenamente a solução.	O README.md apresenta de forma clara a solução, sua arquitetura, instruções de configuração e execução, principais decisões técnicas, cenários e evidências necessárias para compreender, reproduzir e avaliar o projeto.

Aplicação, arquitetura agêntica e integrações

Nº	Critério de Avaliação	0	0,25	0,75
6	A aplicação está funcional e demonstra adequadamente o problema e os cenários definidos?	A aplicação não executa, depende exclusivamente de respostas fixas ou não permite compreender o problema que pretende resolver.	A aplicação executa parcialmente ou demonstra apenas um dos cenários esperados, apresentando limitações relevantes no fluxo.	A aplicação executa de ponta a ponta, possui problema e domínio claramente definidos, demonstra os dois cenários esperados e produz uma saída estruturada e adequada ao domínio.
7	O fluxo foi modelado adequadamente com LangGraph?	Não há LangGraph funcional ou o fluxo não possui state, nodes e edges compreensíveis.	O grafo existe, mas possui estado confuso, nodes excessivamente acoplados ou controle de fluxo incompleto.	O grafo possui state tipado, nodes claros, execução sequencial, ramificação condicional, paralelização simples, condição de parada e separação coerente entre decisões do modelo e regras determinísticas.
8	A solução utiliza adequadamente uma tool integrada?	Não há tool funcional ou integração demonstrável.	A tool existe, mas apresenta integração, validação ou tratamento de falhas incompleto.	A solução possui uma tool funcional integrada por MCP, API, serviço, backend ou webhook, com validação e tratamento de falhas.
9	A solução utiliza uma estratégia de memória ou recuperação contextual adequada ao domínio?	Não há estratégia de memória ou recuperação contextual, ou o contexto é inserido de forma fixa e descontrolada.	Existe uma estratégia de contexto, memória ou recuperação, mas sua utilização é limitada, pouco documentada ou	A solução utiliza de forma coerente uma estratégia de memória ou recuperação contextual adequada ao domínio, como state , checkpoint,

			não demonstra claramente o uso de informações relevantes pela aplicação.	armazenamento persistente ou RAG, recuperando e utilizando informações relevantes conforme a necessidade da aplicação.
--	--	--	--	--

Segurança, observabilidade e resiliência

Nº	Critério de Avaliação	0	0,25	0,75
10	A solução aplica controles adequados de segurança e limites de autonomia?	Não há proteção adequada de segredos, validação das ações ou tratamento de entradas não confiáveis.	Existem controles de segurança, mas estão incompletos ou o cenário adversarial não demonstra claramente o comportamento seguro da aplicação.	A solução protege credenciais, valida ações e entradas e demonstra um cenário adversarial no qual ações não autorizadas são bloqueadas. Aprovação humana é utilizada quando aplicável ao domínio.
11	A execução possui pelo menos dois sinais de observabilidade correlacionados e tratamento adequado de falhas?	Não há sinais suficientes para investigar a execução e as falhas não são tratadas adequadamente.	Há apenas um sinal, os sinais não possuem correlação suficiente ou o tratamento de falhas está incompleto.	A solução produz e correlaciona pelo menos dois sinais de observabilidade, incluindo logs estruturados, e permite identificar decisões, erros e latência. Também trata falhas com timeout, retry ou fallback, quando necessário.

QA, DevOps Inteligente e Low-Code

Nº	Critério de Avaliação	0	0,25	0,50
12	O estudante aplicou IA em code review e testes relevantes?	Não há evidência relevante de uso de IA em revisão de código ou testes.	Há code review ou testes gerados, mas com cobertura superficial, sem validação crítica ou priorização por risco.	A IA é utilizada em code review de uma alteração real e na geração ou refinamento de testes relevantes, incluindo pelo menos um dos seguintes tipos: integração, aceitação ou E2E, com priorização por risco ou impacto.
13	A solução contempla DevOps inteligente, detecção de anomalias e estimativa de	Não há pipeline funcional nem análise de logs, anomalias ou	Há pipeline e uma análise parcial, mas faltam etapas, sinais, evidências ou	O pipeline executa lint, testes e build ou validação equivalente; a IA explica logs de pelo menos duas etapas,

	tendência ou risco de falha?	estimativa de risco de falha.	interpretação estruturada.	detecta e explica uma anomalia e produz uma estimativa simples de tendência ou risco. O deploy é considerado apenas quando aplicável.
14	O estudante integrou uma automação low-code ou no-code ao projeto?	Não há fluxo low-code/no-code demonstrável.	Existe um fluxo visual isolado, incompleto ou sem integração real com a aplicação.	Há automação low-code ou no-code integrada à solução principal, com trigger, integração com a aplicação ou serviço, saída observável e instruções de reprodução.

Análise crítica e evidências				
Nº	Critério de Avaliação	0	0,25	0,50
15	O estudante documentou um refinamento relevante da solução e apresentou evidências do desenvolvimento?	Não há análise crítica ou evidências de refinamento.	Há menção a um problema ou alteração, mas sem mostrar claramente a decisão e o resultado.	O estudante apresenta o problema ou comportamento observado, a alteração realizada, a justificativa, o resultado obtido e evidências relacionadas ao desenvolvimento.

TOTAL: 10,00 pontos

7. CHECKLIST FINAL DE ENTREGA

Antes de submeter no AVA, confira:

Repositório e organização:

- ☐ Criei o repositório no GitHub, adicionei o professor como colaborador e confirmei que nenhum segredo ou arquivo **.env** foi versionado.
- ☐ Organizei e mantive atualizado o quadro Kanban durante o desenvolvimento.
- ☐ Utilizei o fluxo **develop** → **feature/*** → **develop** → **main**, com commits semânticos e coerentes com a evolução do projeto.
- ☐ Mantive na **main** a versão final e funcional da aplicação.

Domínio, arquitetura e agente:

- ☐ Defini o problema, o domínio e demonstrei dois cenários de uso, incluindo um cenário de risco, falha, exceção ou anomalia.

- ☐ Implementei o fluxo principal com LangGraph, incluindo state, nodes, execução sequencial, ramificação condicional, paralelização e condição de parada.
- ☐ Implementei uma tool funcional integrada por MCP, API, serviço, backend ou webhook;
- ☐ Adotei uma estratégia de memória ou recuperação contextual adequada à aplicação.

Segurança, observabilidade e resiliência:

- ☐ Apliquei controles de segurança, incluindo validação de payloads, parâmetros, schemas e permissões, limites de autonomia, aprovação humana quando necessária e um cenário adversarial de prompt injection ou entrada não confiável.
- ☐ Produzi e correlacionei pelo menos dois sinais de observabilidade, sendo logs estruturados e um segundo sinal entre trace, métrica ou auditoria, registrando informações como erros e latência..
- ☐ Tratei falhas nas integrações, utilizando timeout, retry limitado ou fallback quando necessário.

QA, DevOps e Low-Code:

- ☐ Realizei code review com apoio de IA e gerei ou refinei testes relevantes, incluindo pelo menos um dos seguintes tipos: integração, aceitação ou E2E, com priorização baseada em risco, impacto ou criticidade;
- ☐ Configurei pipeline com lint, testes e build ou equivalente e utilizei IA para analisar logs de pelo menos duas etapas, detectar uma anomalia e produzir uma estimativa simples de tendência ou risco de falha;
- ☐ Integrei uma automação low-code/no-code à solução principal, com trigger e saída observável.

README.md e evidências:

- ☐ O README.md permite compreender, configurar, executar e avaliar a solução, incluindo as principais instruções do agente e a configuração do modelo por variável de ambiente.
- ☐ Documentei pelo menos um ciclo de refinamento, apresentando o problema observado, a alteração realizada e o resultado obtido.
- ☐ Organizei as principais evidências de testes, observabilidade, QA, DevOps e low-code/no-code;

- ☐ Incluí no README.md o link do vídeo de demonstração.

Vídeo e submissão:

- ☐ Gravei o vídeo com duração recomendada de até 10 minutos, sem ultrapassar 12 minutos, e publiquei como não listado.
- ☐ Demonstrei os dois cenários e os principais artefatos técnicos, incluindo pipeline, análise de logs, anomalia, estimativa de tendência ou risco e fluxo low-code/no-code.
- ☐ Mantive no repositório e no quadro do GitHub as evidências necessárias para acompanhar e avaliar o desenvolvimento individual.
- ☐ Submeti no AVA os links do repositório, quadro e vídeo antes do prazo e não alterei o repositório após a entrega.